

Parameter Efficient Finetuning

Too Expensive GPUs and Too Large LLMs



	Qwen2-72B Instruct	Llama3-70B Instruct	Qwen1.5-72B Chat
MMLU	82.3	82.0	75.6
MMLU-Pro	64.4	56.2	51.7
GPQA	42.4	41.9	39.4
TheoremQA	44.4	42.5	28.8
HumanEval	86.0	81.7	71.3
MBPP	80.2	82.3	71.9
MultiPL-E	69.2	63.4	48.1
LiveCodeBench	35.7	29.3	17.9
GSM8K	91.1	93.0	82.7
MATH	59.7	50.4	42.5
MT-Bench	9.12	8.95	8.61
MixEval	86.7	84.0	84.1
Arena-Hard	48.1	41.1	36.1
IFEval	77.6	77.3	55.8
AlignBench	8.27	7.42	7.28

**Is it possible to fine-tune a ~70B model
with only a single 48 GB GPU?**

Preliminary Analysis of GPU Memory Usage in PyTorch

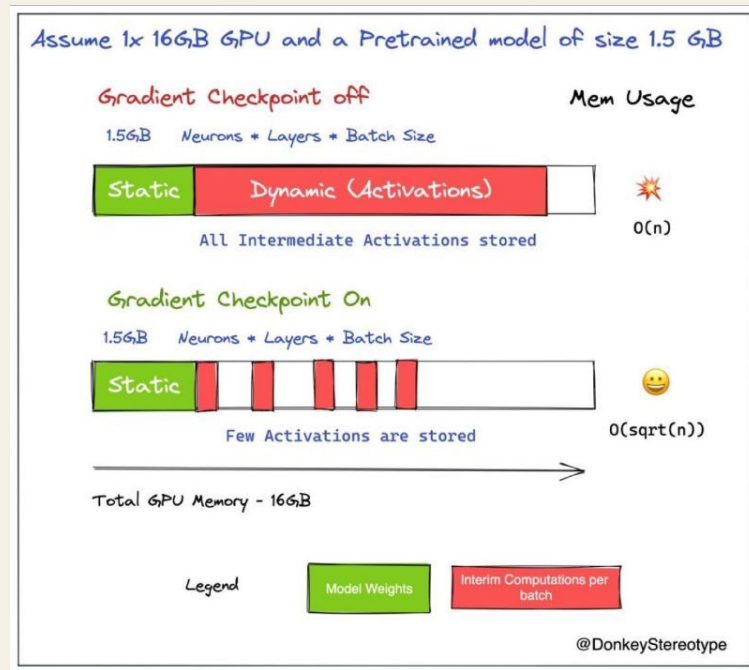
- Training only a subset of parameters does not reduce model parameter or activation memory.
- PEFT reduces memory usage for gradients and optimizer states.



Reducing Memory Cost

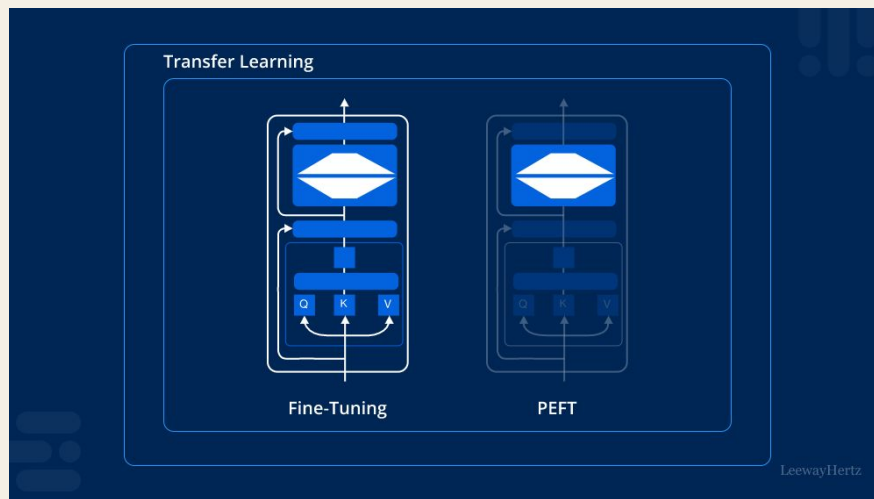
- Model parameters: Not dominant in current full fine-tuning settings.
- Activation memory:
 - Reduce batch size
 - Use gradient checkpointing
 - (Both trade memory for increased computation time.)
- Gradient and optimizer states:

Parameter Efficient Finetuning



Recap of PEFT

- Full fine-tuning updates every parameter in the Transformer.
- Fine-tuning all parameters of large language models is expensive in memory, compute, and storage.
- PEFT methods adapt models by training only a small number of parameters while freezing the backbone.



Adapters (Houlsby et al., 2019)

- Adapters insert small trainable bottleneck layers into each Transformer block.
- The original model weights stay frozen, while only adapter weights are updated.
- This enables modular multi-task tuning with low additional parameters.

```
def adapter(x):  
    z = W_down(x)           # d_model -> d_adapter  
    z = relu(z)  
    z = W_up(z)             # d_adapter -> d_model  
    return z  
  
def transformer_block_with_adapter(x):  
    x = x + attention(layer_norm(x))  
    x = x + mlp(layer_norm(x))  
    x = x + adapter(x)      # train only adapter weights  
    return x
```

Adapter Training Setup

- Adapter-based PEFT works by freezing attention/MLP weights.
- Only small adapter modules contribute gradients and update during training.
- Each downstream task stores only the adapter weights instead of a full model.
- General way of implementing PEFT.



```
def set_trainable_adapters_only(model):  
    for name, p in model.named_parameters():  
        p.requires_grad = ("adapter" in name)
```

Prefix Tuning (Li & Liang, 2021)

- Prefix tuning prepends trainable “virtual tokens” to the attention key/value states.
- The Transformer weights remain frozen, but attention sees additional learned context.
- This works especially well for generation tasks.



```
def prefix_attention(q, k, v, prefix_k, prefix_v):  
    k = concat(prefix_k, k, dim=2)  
    v = concat(prefix_v, v, dim=2)  
  
    attn = softmax(q @ k.transpose(-2, -1))  
    return attn @ v
```

Prefix Parameters (How Prefix is Produced)

- Prefix tuning learns separate prefix vectors per layer (often per head).
- These vectors act like “extra memory” injected into attention.
- At inference, prefix vectors can be loaded cheaply per task.



```
def get_prefix_kv(prefix_params, batch_size):  
    prefix_k = prefix_params.K.repeat(batch_size, 1, 1, 1)  
    prefix_v = prefix_params.V.repeat(batch_size, 1, 1, 1)  
    return prefix_k, prefix_v
```

Prompt Tuning (Lester et al., 2021)

- Prompt tuning freezes the entire model and trains only a small set of embedding vectors.
- These embeddings behave like a learned continuous prompt prepended to the input.
- It is extremely parameter-efficient but can be less expressive than LoRA/adapters.

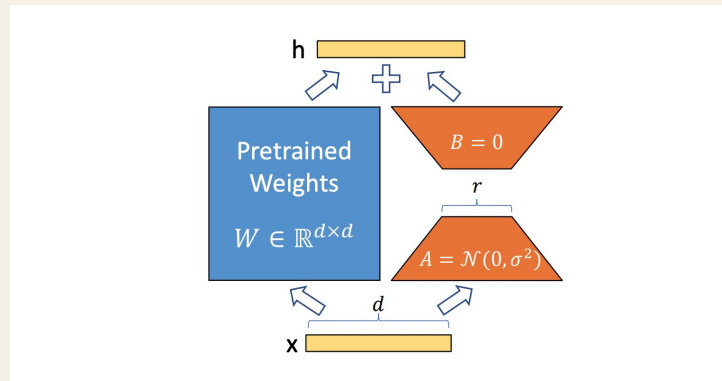
```
def prompt_tuning_forward(token_ids, soft_prompt_ids):  
    prompt_emb = soft_prompt_table(soft_prompt_ids)  
    token_emb = token_embedding(token_ids)  
  
    x = concat(prompt_emb, token_emb, dim=1)  
    return transformer(x)
```

Limitations of Previous PEFT Methods

- **Adapters:** Introduce additional layers into the network, increasing inference latency and computational cost.
- **Prefix Tuning:** Adds learned key/value tokens at every layer, increasing context length and attention computation.
- **Prompt Tuning:** Relies only on input embeddings, limiting expressivity and often requiring longer prompts to achieve strong performance.

LoRA (Hu et al., 2021)

- LoRA represents weight updates as a low-rank matrix decomposition.
- Instead of updating full attention/MLP matrices, it learns two small matrices A and B.
- This gives strong adaptation with very few trainable parameters.



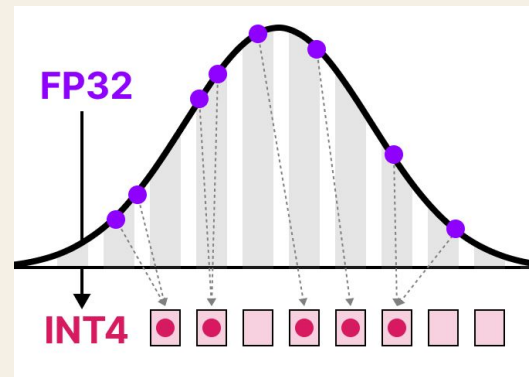
LoRA in Attention Projections

- In practice, LoRA is applied to large projection matrices like W_q , W_k , W_v , and W_o .
- The base weights remain frozen, so multiple LoRA adapters can share one backbone model.
- After training, LoRA weights can be merged into the base model for deployment (same inference cost).

```
def lora_linear(x, W, A, B):  
    # W frozen, A and B trainable  
    delta_W = B @ A  
    return x @ (W + delta_W).T  
  
def attention_with_lora(x):  
    q = lora_linear(x, Wq, Aq, Bq)  
    k = lora_linear(x, Wk, Ak, Bk)  
    v = lora_linear(x, Wv, Av, Bv)  
  
    attn = softmax(q @ k.transpose(-2, -1))  
    return attn @ v
```

Reduce Model Parameters Memory

- Previously, we said model parameters were not the main memory bottleneck.
- However, once we apply gradient accumulation, gradient checkpointing, and LoRA, the memory usage of other components becomes much smaller.
- As a result, model parameter memory becomes dominant again.
- How can we reduce this? **Quantization.**



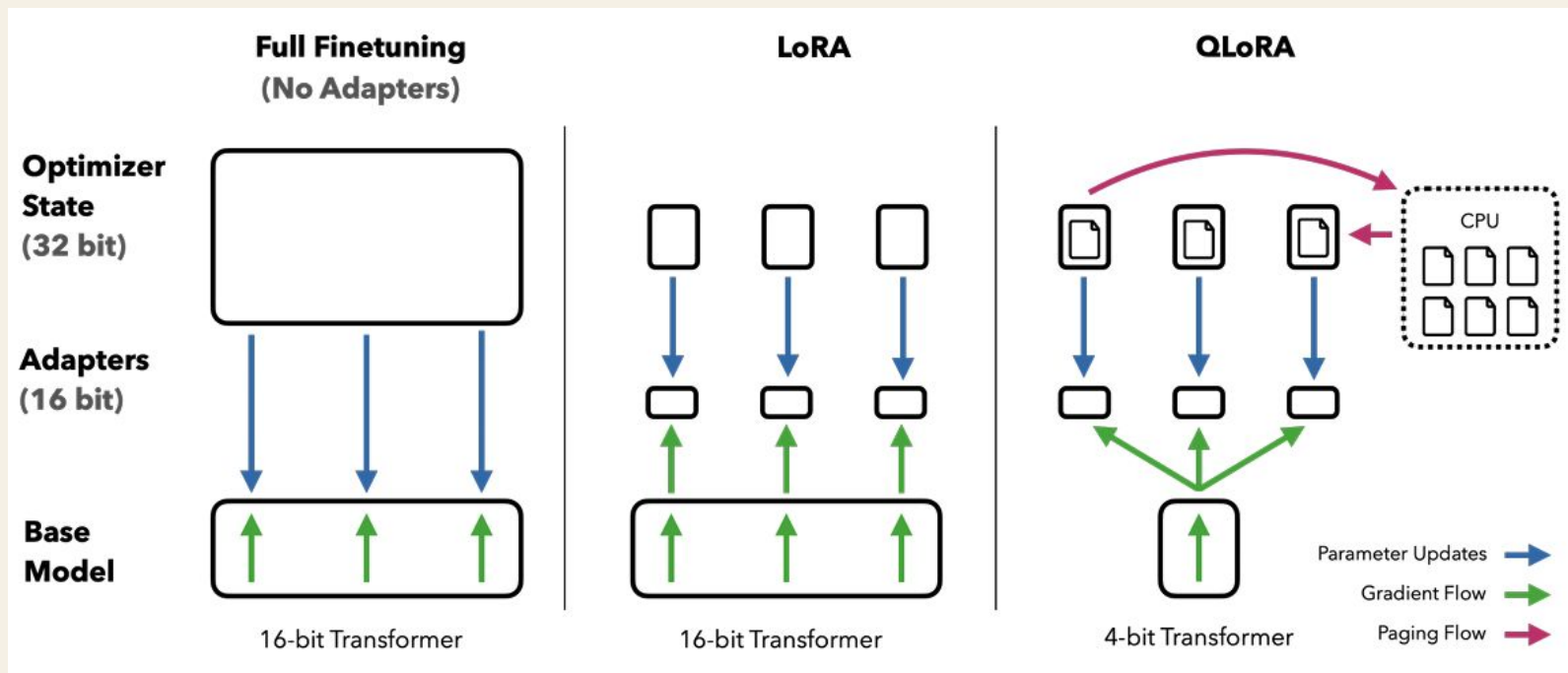
QLoRA (Dettmers et al., 2023)

- QLoRA quantizes the frozen backbone model weights to 4-bit (new datatype NormalFloat) to save memory.
- LoRA adapters are trained in higher precision while gradients flow through quantized weights.
- This enables fine-tuning very large models on a single GPU.



```
def qlora_linear(x, W_4bit, A, B):  
    W = dequantize_4bit(W_4bit)      # compute-time dequantization  
    delta_W = B @ A                  # LoRA update in fp16/bf16  
    return x @ (W + delta_W).T
```

QLoRA



Finetune 70B model with 48GB GPU

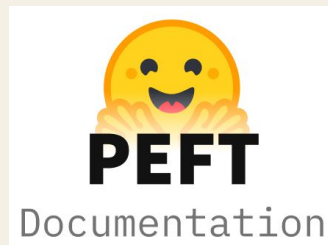
Hardware Requirement

** estimated*

Method	Bits	7B	13B	30B	70B	110B	8x7B	8x22B
Full	AMP	120GB	240GB	600GB	1200GB	2000GB	900GB	2400GB
Full	16	60GB	120GB	300GB	600GB	900GB	400GB	1200GB
Freeze	16	20GB	40GB	80GB	200GB	360GB	160GB	400GB
LoRA/GaLore/BAdam	16	16GB	32GB	64GB	160GB	240GB	120GB	320GB
QLoRA	8	10GB	20GB	40GB	80GB	140GB	60GB	160GB
QLoRA	4	6GB	12GB	24GB	48GB	72GB	30GB	96GB
QLoRA	2	4GB	8GB	16GB	24GB	48GB	18GB	48GB

PEFT Library

- 🙌 PEFT (Parameter-Efficient Fine-Tuning) is a library for efficiently adapting large pretrained models to various downstream applications without fine-tuning all of a model's parameters because it is prohibitively costly.
- PEFT is integrated with the Transformers, Diffusers, and Accelerate libraries to provide a faster and easier way to load, train, and use large models for inference.



PEFT Library: LoRA in Practice

- PEFT wraps an existing Transformer and injects LoRA layers into target modules.
- Only LoRA parameters are trainable, keeping the backbone frozen.
- This enables fast fine-tuning while reusing the same pretrained weights.

```
from peft import LoraConfig, get_peft_model

config = LoraConfig(
    r=8,
    lora_alpha=16,
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.05
)

model = get_peft_model(model, config)
model.print_trainable_parameters()
```

PEFT Library: Save and Merge Adapters

- PEFT adapters can be saved separately, making them lightweight to distribute.
- Adapters can be loaded dynamically for multi-task inference.
- `merge_and_unload()` merges LoRA weights into the backbone for deployment.



```
model.save_pretrained("lora_adapter")

from peft import PeftModel
model = PeftModel.from_pretrained(model, "lora_adapter")

merged_model = model.merge_and_unload()
```

Thank you!

Any questions?